LAB EXERCISE 3

Improve URL Shortener Performance Using Redis Caching

AIM

To improve the URL Shortener application built in Experiment 2 by adding Redis as an in-memory cache in front of the database, using the cache-aside strategy, and to compare the response time of retrieving a URL with and without the cache.

What This Experiment Actually Does (In Simple Terms)

In Experiment 2, every time someone opened a short URL, the app had to ask the H2 database for the original long URL. A database lookup is not instant — it takes some time. Redis (here running as **Memurai**, a Windows-compatible build of Redis) is an in-memory key-value store: it keeps data directly in RAM instead of on disk, so reading from it is much faster than reading from a database.

The idea used here is called the **cache-aside pattern**. It just means: check the cache first, and only go to the database if the cache does not already have the answer.

Step by step, for GET /{shortCode}:

1. The app checks Redis first, using the short code as the key.

2. **Cache Hit** — if Redis already has that short code stored, the long URL is returned immediately from Redis. The database is never touched.

3. **Cache Miss** — if Redis does not have it (e.g. the very first time this short code is opened), the app falls back to the database exactly like Experiment 2 did.

4. After a cache miss, before returning the answer, the app saves that short code and long URL into Redis. So the next person who opens the same short link gets a cache hit.

In short: the first visit to a link is a little slow (database), but every visit after that is fast (Redis) — until the cache is cleared or Memurai restarts, since it is an in-memory store.

What Changed From Experiment 2

This experiment reuses the Experiment 2 project as-is and adds only what the Redis caching requirement needs. Nothing else was rewritten.

File	Change
pom.xml	Added one Maven dependency: `spring-boot-starter-data-redis`.
application.properties	Added `spring.data.redis.host` and `spring.data.redis.port` so Spring Boot knows where Memurai is running (localhost:6379).
UrlShortenerController.java	Injected a `StringRedisTemplate` and rewrote the `GET /{shortCode}` method to check Redis before the database, and to write the result into Redis on a cache miss. `POST /shorten` is unchanged.
Every other file	UrlMapping.java, UrlMappingRepository.java, ShortenRequest.java, ShortenResponse.java, UrlShortenerApplication.java — copied over from Experiment 2 with no edits.

System Architecture

A client (Postman or a browser) sends a request to the Spring Boot app. For a GET request, the controller now checks Redis before it checks the H2 database. Redis and H2 both sit behind the same controller; the client never talks to either one directly.

```
Client (Browser / Postman)
        |
        v
UrlShortenerController
        |
        |--- GET: check Redis first ---> Redis (Memurai) --- hit ---> return long URL
        |                                    |
        |                                  miss
        |                                    v
        |--- fall back to ---------------> H2 Database
        |                                    |
        |<---- store result in Redis ---------+
        |
        |--- POST: save mapping -----------> H2 Database (unchanged from Exp 2)
```

REST API (Same Endpoints as Experiment 2)

Method	Endpoint	Description
POST	/shorten	Accepts `{ "longUrl": "..." }` and returns HTTP 201 with the generated short URL. Behaviour unchanged from Experiment 2.
GET	/{shortCode}	Checks Redis first (cache-aside). Returns HTTP 302 redirecting to the long URL on a hit or a miss, and HTTP 404 if the short code does not exist at all.

SOURCE CODE

pom.xml (added dependency shown in bold context below)

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- new for Experiment 3 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

application.properties (new lines at the bottom)

```
server.port=8080

# In-memory H2 database - no installation needed, resets when app stops
spring.datasource.url=jdbc:h2:mem:urlshortenerdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Lets Hibernate create the table automatically from UrlMapping.java
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Optional: view the database in a browser at http://localhost:8080/h2-console
spring.h2.console.enabled=true
```

```
# Redis - running locally via Memurai on the default port
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

UrlShortenerController.java (full file)

```
// heart of the project
// contains two rest endpoint
// post/shorten - to generate short url
// get/{shortcode} -  controller search the db if found rediect 302 else 404 not found


// take google.com
// generate sha 256hash - why sha same input - same output,
// fast,diffuclt to rever,gnereate unique looking values


// covert into base 64
// why base 64:
//     sha is hexadecimal ie 0-9 a-f
//     but base:
//     A-Z
//     a-z
//     0-9
//     so it create shorther readble ones

// take first 6 charcaters
// check db whether it exsit
// if exsit:
//     Ayirwc
// else:
//     AyirwcCx or AyirwcC


// databse:
//     table name = url_mapping
//     colums
//         id          =1
//         shortCode   =airwyc
//         longUrl     = https://google.com

// http status code
// post 201 created
// get 302 found
// 404 not found means not exist




package com.ssn.urlshortener;

import org.springframework.data.redis.core.StringRedisTemplate;
// spring boot auto creates this bean once redis starter is on classpath, no config class
  needed
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

// import java.net.URI;
import java.nio.charset.StandardCharsets;
// Used while converting text into bytes before hashing.
import java.security.MessageDigest;
// used to genereate sha -256
import java.util.Base64;
// to covert the hash into base64
```

```
import java.util.Optional;
// may or may not find a url



@RestController //class handles http requests
public class UrlShortenerController { //controller

    private final UrlMappingRepository repository; //stores repositry olbject which is used
  to commiuncate with the db
    private final StringRedisTemplate redis; //talks to memurai/redis, key = shortcode
  value = longurl

    //construcor
    public UrlShortenerController(UrlMappingRepository repository, StringRedisTemplate
  redis) {
        this.repository = repository;
        this.redis = redis;
    }


    @PostMapping("/shorten") //first api
    public ResponseEntity<ShortenResponse> shorten(@RequestBody ShortenRequest request) {

        String longUrl = request.getLongUrl();
        String shortCode = generateUniqueShortCode(longUrl);
        //calls a function to generate a shortcode

        UrlMapping mapping = new UrlMapping(shortCode, longUrl);
        repository.save(mapping);

        String shortUrl = "http://localhost:8080/" + shortCode;
        ShortenResponse response = new ShortenResponse(shortUrl);

        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }


    @GetMapping("/{shortCode}")
    public ResponseEntity<Void> redirect(@PathVariable String shortCode) {

        // cache-aside: check redis first before touching the db
        String longUrl = redis.opsForValue().get(shortCode);

        if (longUrl != null) {
            // cache hit, straight from redis, no db call
            return ResponseEntity.status(HttpStatus.FOUND)
                    .header(HttpHeaders.LOCATION, longUrl)
                    .build();
        }

        // cache miss, fall back to db like before
        Optional<UrlMapping> result = repository.findByShortCode(shortCode);

        if (result.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        longUrl = result.get().getLongUrl();

        // now store it in redis so next request for this shortcode is a hit
        redis.opsForValue().set(shortCode, longUrl);

        return ResponseEntity.status(HttpStatus.FOUND)
                .header(HttpHeaders.LOCATION, longUrl)
                .build();
```

```
//returns
//Status

// 201 Created

// Body

// {
// "shortUrl":
// "http://localhost:8080/Ayirwc"
// }
    }


    private String generateUniqueShortCode(String longUrl) {

        String hash = sha256Base64(longUrl);
        int length = 6;
        String code = hash.substring(0, length);

        while (repository.findByShortCode(code).isPresent() && length < 8) {
            length++;
            code = hash.substring(0, length);
        }

        return code;
    }

    private String sha256Base64(String input) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            //creates sha-256 hashing object
            //sha256 works on byte not strings
            //utf-8bytes - > sha256-  hashbytes
            byte[] hashBytes = digest.digest(input.getBytes(StandardCharsets.UTF_8));

            return Base64.getUrlEncoder().withoutPadding().encodeToString(hashBytes);

        } catch (Exception e) {
            throw new RuntimeException("Error generating hash", e);
        }
    }
}
```

UrlMapping.java, UrlMappingRepository.java, ShortenRequest.java, ShortenResponse.java and UrlShortenerApplication.java are identical to Experiment 2 and are not repeated here.

VERIFICATION (application run for real, with real requests)

The app was built and run locally with `mvn spring-boot:run` (port 8080), with Memurai already running as a Windows service on port 6379. Every request below was sent with curl and its real response is shown, including curl's own `time_total` timer for measuring response time.

```
=== Request 1: POST /shorten ===
curl -i -X POST http://localhost:8080/shorten -H "Content-Type: application/json" \
    -d '{"longUrl": "https://www.google.com/search?q=redis+caching"}'

HTTP/1.1 201
Content-Type: application/json
{"shortUrl":"http://localhost:8080/a2KA_d"}


=== Request 2: GET /a2KA_d - first time (cache MISS, falls back to H2) ===
curl -i http://localhost:8080/a2KA_d
HTTP/1.1 302
```

```
Location: https://www.google.com/search?q=redis+caching
time_total: 0.688304s


=== Request 3: GET /a2KA_d - second time (cache HIT, straight from Redis) ===
curl -i http://localhost:8080/a2KA_d

HTTP/1.1 302
Location: https://www.google.com/search?q=redis+caching
time_total: 0.004213s


=== Request 4: GET /a2KA_d - third time (still a cache HIT) ===
curl -i http://localhost:8080/a2KA_d

HTTP/1.1 302
Location: https://www.google.com/search?q=redis+caching
time_total: 0.018217s


=== Request 5: GET /notreal99 - short code that was never created ===
curl -i http://localhost:8080/notreal99

HTTP/1.1 404
time_total: 0.005732s


=== Redis check - confirming the value is really sitting in the cache ===
memurai-cli.exe GET a2KA_d

"https://www.google.com/search?q=redis+caching"
```

Request 5 shows the cache-aside logic does not break the existing 404 behaviour: an unknown short code is a cache miss, then a database miss too, so it still correctly returns 404 without ever storing anything in Redis.

Performance Comparison

Retrieval	Source	Response Time
Request 2 (1st GET)	H2 database (cache miss)	688 ms
Request 3 (2nd GET)	Redis / Memurai (cache hit)	4.2 ms
Request 4 (3rd GET)	Redis / Memurai (cache hit)	18.2 ms

The first request (cache miss) went through the full path: Redis check, then a JPA/H2 query. The second request (cache hit) skipped the database completely and returned in a fraction of the time. The single-run numbers above vary between requests (688ms/4.2ms/18.2ms) because this is one JVM warming up on a shared machine, not a controlled benchmark — but the pattern is consistent and clear in every run: the database path is always tens to hundreds of times slower than the Redis path, exactly the improvement Redis caching is supposed to give.

Tasks Performed

#	Task	Result
1	Configured Redis with Spring Boot	Added the Redis starter dependency and host/port config; Spring Boot auto-created a StringRedisTemplate bean, no manual config class needed.

2	Cached URL mappings by short code	Redis key = short code, value = long URL, e.g. a2KA_d -> https://www.google.com/search?q=redis+caching.
3	Implemented cache-aside in GET /{shortCode}	Checks Redis first; on a miss, reads H2 then writes the result into Redis.
4	Tested with curl in place of Postman	Ran 5 real HTTP requests against the running app (Requests 1-5 above).
5	Measured response time with and without Redis	Cache miss: 688ms. Cache hits: 4.2ms and 18.2ms.
6	Verified the cache directly	Ran memurai-cli GET a2KA_d and confirmed the exact long URL was stored, not just trusting the HTTP response.
7	Confirmed 404 behaviour was not broken	GET /notreal99 still correctly returns 404 after adding the cache layer.

Learning Outcomes

1. I was able to integrate Redis into an existing Spring Boot application without changing its public API.

2. I understood and implemented the cache-aside caching strategy: check cache, fall back to the database on a miss, then populate the cache.

3. I learned why an in-memory store like Redis is faster than a relational database for simple key-value lookups — it avoids disk I/O and query parsing entirely.

4. I was able to measure and compare real response times for a cache hit versus a cache miss using curl.

5. I understood the difference between the two failure/skip paths in cache-aside: a cache miss (falls back to DB) versus a true 404 (not found anywhere, DB included).

6. I learned that a cache is only as good as what is in memory: since Memurai is in-memory, restarting it clears everything, unlike the data stored in the database.